Kutty Banerjee

100 Palm Valley #3058 San Jose CA 95123
(925)-209-7204 kutty.banerjee@gmail.com

Expertise System Software, GPU Drivers, SW/HW GPU Design, Parallel Runtimes. Worked on a Full GPU Architecture life cycle and 2 partial life cycles.

Graphics Software Architect, Intel [June 2013 ~ Present]

- Drive SW/HW changes for DirectCompute on Next Generation CellPhone, Tablets.
- Power Efficient Shared Virtual Memory Design.

Software Engineer, NVIDIA [May 2005 ~ May 2013]

- **MTS Just-in-Time Compiler Runtime/Microkernel**
  - Maintained the Trace-Profiler, Trace-Cache, Garbage Collector, Runtime for a **JIT** /**Microkernel** called MTS that translates **ARM64** to VLIW for a native **NVIDIA CPU**.
  - Added GCC attributes lockpath, lock, lockaccess to statically catch lock composition errors
  - Patched GCC attribute rules to prevent ISR code from invoking non ISR high latency code
  - Patched GDB to implement LockStat and Auto Deadlock Detection.
  - Implemented Lazy Branch-asserts to improve code-scheduling by removing cold branches.
  - Implemented Branch-Reprofiling to capture Program Profile changes mid-execution
  - Patched Microkernel Scheduler and interrupt code to prevent Garbage Collector starvation.
- **Android FullStack CPU Optimizations**
  - Implemented Large Page Support for ARM Linux by coalescing adjacent contiguous pages into a Super Page. Principle similar to BSD Super Pages.
  - Implemented a Memory Profiler on top of QEMU to monitor CPU Page Access Patterns.
  - Implemented a Thread Local Memory type in Linux kernel. TLB invalidates, cache snooping are disabled on such pages until Threads migrate. Helps Google V8 benchmark IPC by 20%
  - Patched Mozilla, V8 Browser, Linux kernel to take advantage of our CPU's I/D coherence.
- **CUDA Shared Virtual Memory**
  - Lead SW/HW Uniform Address Space implementation efforts between CPU/GPU for a Cache-Coherent SoC design targeting Super Computers.
  - Designed ARM64 Linux OS changes and GPU kernel changes for Single Address Space b/w CPU/GPU using Shadow Page Tables. Changes included keeping Page Tables, TLBs, Access-Bit information coherent b/w CPU and GPU purely in SW.
  - Did Performance profiling of GPU TLB invalidate for victim and initiator latencies under various workloads. Did similar studies for GPU interrupt routing latencies for fault-passing to CPU using on board micro controller. Project was eventually canceled.
- **SW GPU/Chipset Verification Lead**
  - The role was to reach out to the various HW unit leads, SW stake holders and make sure we had a coherent Chip Verification Strategy across HW/SW for future GPUs/Chipsets.
  - Improved Simulation Framework to playback real world test cases using Directed Tests.
  - Designed Probe-Based testing Framework to test Driver Interrupt Code and its interaction with Asynchronous GPU events. Idea is to place Probes in the driver and ISR code. When activated, probes can add arbitrary delays/waits for certain conditions thereby forcing determinism and reproducibility. Once reproduced, logic correctness can be tested.
  - Put in place Code Coverage System for Driver code using Bullseye.
- **Fermi Architecture Driver Chip Lead (Resource Manager Team)**
  - Lead the overall technical efforts for Fermi Generation GPUs for the Resource Manager Team (OS Team) from Functional Simulation, Microarchitecture Feature Design through RTL/Emulation and Bringup.
  - Was the main contact for debug and interaction with OGL/DX/CUDA/HW teams.
- **Fermi Command Buffer Scheduling (Software Component Lead)**

- Worked closely OGL/DX/CUDA/HW teams to identify hotspots, pitfalls from previous generation and drive HW changes to fix them. Rewrote this code in the driver for Fermi.
- Previous architectures had a single instruction fetch/decode/feed-engine pipeline which was unable to saturate PCIe bandwidth or keep GPU engines busy. Implemented SW support to split/ schedule instruction streams across separate Instruction Pipelines and tweak timeslicing based on work load pattern to keep PCIe fetch and engine feed optimally busy.
- **Pipelined Scheduling** Drove efforts to implement a new scheduling mode tuned for Vista WDDM with full HW support. This allowed OS to queue work on various engines in order without explicit synchronization. Added efficient notifier support. Added support code in driver for new mode.
- Refactored code in SW that maintained Schedulable work called Runlists. Split global runlists into **Multiple Per Engine Runlists** to avoid concurrency problems between Compute / Graphics and Copy Engines. Implemented double buffering to allow CPU to modify runlist without stopping GPU. Implemented an indexed bitmap for the data structure to get better CPU cache locality during Runlist searches.
- **Efficient Software Method Handling** Software methods (traps) allow pipelined execution of CPU code with GPU code. Implemented this handling to allow a bulk processing of SW methods from a single interrupt to reduce interrupt latency. Also added code for out or order SW method execution when SW methods are interleaved with hw methods.
- **Power Aware Scheduling (ELPG)** This existing feature allows engines to power down during bursts of inactivity. My role here was to port this code to the new scheduler and make sure stop scheduling and revive scheduling of power gated engines worked correctly. This feature was later staffed by an entire team and moved to microcode, helped them in this process.
- **Workload Metering** Throttle incoming OS work to allow engines to be in deeper power saving state for longer. Did initial scoping for SW/HW changes. Feature was postponed.
- **Fault Recovery** Worked closely with the hw team to design SW for recovery from page faults engine hangs and other conditions and identify changes required for new scheduling modes. Actual implementation was done by other members in my team
- **Virtual Thread** Provided technical guidance for implementing large number of Threads in SW on top of HW Threads by efficient swapping. Code was written by another team member.
- **Pipelined Page/Resource Swapping** Worked with DX/HW to implement resource stealing from contexts that are not currently resident on the GPU and restoring them back right before they get to run. My contribution was injecting surrender resource / reclaim resource methods in existing instruction streams.

- **Vista Non Stall Interrupts and Lockfree ISRs**
  - Worked closely with hw to design a new interrupt tree for handling non stalling interrupts. Rewrote a separate ISR that has minimal data dependency on the main ISR and rest of driver. This was designed to deliver Notifiers to the OS w/o waiting or attempting to lock since this can effect frame rate on Vista.
- **Self Modifying Code**
  - Implemented API and designed sequence with HW for OpenGL/CUDA/DX clients to use to modify their instruction stream after it has been queued in the GPU without introducing massive stalls. Under the hood this mechanism injects noops and acquire/releases and pipelined waits in the method stream before blitting in new code.
- **Mid-Buffer Preemption**
  - The goal here is to allow modification/reordering queued work and be able to inject new work with minimal latencies. Useful for desktop interactivity. My contribution was to provide

APIs for Vista driver teams to use that allow submitted but not yet executed buffers to be reordered. Leverages Self Modifying Code Support from above

- **Fermi NOMMU Support**
  - Modified heap to allow physical and virtual addresses to match. This is very useful for debugging a new MMU HW during Simulation and Bringup.
- **Fermi MMU Page Table Support**
  - Implemented initial code to support two level GPU page table. Helped MMU lead with support for single PDE pointing to two valid page tables. This was for Vista sysmem video memory swapping support. Debugged code.
- **Fermi MMU GPU TLB Invalidate**
  - Wired up the TLB invalidate code. Spent a lot of time debugging issues on Emulation since functional models did not fully support TLB invalidation logic.
- **Fermi MMU NOINVALDATE mode**
  - Modified allocators to skip TLB invalidates and L2 flushes entirely by avoiding virtual address reuse and alignment tricks. This is useful for Perf Simulators that want to quickly race through initialization code and measure a Graphics Frame Trace playback of interest.
- **Fermi GPU Graphics Context Switch**
  - Fermi added a new microcode engine to do this. Helped the component lead find areas to better pipeline save/load operations so they can happen in parallel. Implemented the initial version of context switch code.
- **G72 TurboCache**
  - Helped the MMU component lead make changes in the Heap code to route surfaces to system memory. This was for a low end chip without local video memory.
- **Curie Architecture**
  - Helped debug Power Management, Display, TV, Notebook, Standby Resume, Memory Management issues. Learnt about GPU Driver stacks and driver debugging.

Education

MS Computer Science, Worcester Polytechnic Institute, MA [2003 Aug ~ May 2005]

BS Computer Science, University of Mumbai [1999 July ~ 2003 May]

Publications www.pseudoparallel.com/publications

Side Projects

- Parallel Computing, Locks and Task Parallel Libraries www.pseudoparallel.com

US Permanent Resident